

Sistemas Operacionais

Lista de Exercícios – Capítulo II

Estrutura de um Sistema Operacional

Jeann Victor
17 de outubro de 2025

Disciplina: Sistemas Operacionais

Fonte: Capítulo II – Sistemas Operacionais: Conceitos e Mecanismos

Autor do Material: Prof. Carlos A. Maziero

1 Questão 1

O que diferencia o núcleo do restante do sistema operacional?

O que diferencia o núcleo do restante do sistema operacional é que o núcleo do sistema tem acesso direto ao hardware, sendo assim ele é responsável pela gerência de recursos do hardware e implementa as principais abstrações dos aplicativos e programas utilitários. Outro fator que diferencia o núcleo do restante do sistema é que ele possui um espaço de endereçamento único e exclusivo, enquanto o restante roda em espaços de endereçamento protegidos e isolados.

2 Questão 2

Seria possível construir um sistema operacional seguro usando um processador que não tenha níveis de privilégio? Por quê?

Não seria possível, tendo em vista que não é seguro que todas as aplicações tenham acesso direto ao hardware, pois o acesso direto ao hardware poderia desestabilizar o sistema inteiro, já que poderiam contornar facilmente os mecanismos de controle de acesso aos recursos.

3 Questão 3

Os processadores da família x86 possuem dois bits para definir o nível de privilégio, resultando em 4 níveis distintos. A maioria

dos sistemas operacionais para esses processadores usam somente os níveis extremos (0 e 3, ou 00₂ e 11₂). Haveria alguma utilidade para os níveis intermediários?

Creio que sim, poderia haver algum nível de utilidade em aplicativos que não necessitariam necessariamente de todo o acesso ao recurso do hardware, mas que poderiam precisar de mais acesso do que o nível usuário lhe dá. Entretanto, isso não é necessário tendo em vista que apenas os dois níveis de privilégio já estão satisfazendo, e colocar mais níveis em prática aumentaria muito a complexidade de administração desses níveis, sendo assim não é utilizado.

4 Questão 4

Quais as diferenças entre interrupções, exceções e traps?

Interrupção (ou Interrupção de Hardware): É um sinal assíncrono gerado por um dispositivo de hardware (ex: teclado, disco rígido, placa de rede) para alertar o processador de que um evento ocorreu. Ela pode ser tratada a qualquer momento, independente da instrução que está sendo executada.

Exceção (ou Interrupção Síncrona): É um evento síncrono gerado pela própria CPU como resultado da execução de uma instrução. Exemplos: divisão por zero, acesso a memória inválida, violação de proteção. A exceção está diretamente ligada à instrução que a causou.

Trap (ou Chamada de Sistema): É uma interrupção síncrona e voluntária. O programa executa uma instrução específica com a intenção de solicitar um serviço do sistema operacional. É o mecanismo usado para implementar as chamadas de sistema.

5 Questão 5

O comando em linguagem C `fopen` é uma chamada de sistema ou uma função de biblioteca? Por quê?

O `fopen` na verdade é uma função de uma biblioteca do C, que por baixo dos panos (prepara os parâmetros, invoca a chamada e, no retorno desta, devolve à aplicação os resultados obtidos) faz uma chamada de sistema.

6 Questão 6

A operação em modo usuário permite ao processador executar somente parte das instruções disponíveis em seu conjunto de instruções. Quais das seguintes operações não deveriam ser permitidas em nível usuário? Por quê?

(a) Ler uma porta de entrada/saída

Não deve ser permitida, pois essa operação faz acesso direto ao hardware.

(b) Efetuar uma divisão inteira

Deve ser permitida, é apenas uma operação aritmética.

(c) Escrever um valor em uma posição de memória

Deve ser permitida quando essa escrita é dentro da posição da memória que o processo tem acesso.

(d) Ajustar o valor do relógio do hardware

Não deve ser permitida, tendo em vista que ajustar o valor do relógio do hardware é uma operação de acesso direto ao hardware.

(e) Ler o valor dos registradores do processador

Deve ser permitida.

(f) Mascaram uma ou mais interrupções

Não deve ser permitido, tendo em vista que isso é uma operação crítica de controle do processador (nível núcleo).

7 Questão 7

Considerando um processo em um sistema operacional com proteção de memória entre o núcleo e as aplicações, indique quais das seguintes ações do processo teriam de ser realizadas através de chamadas de sistema, justificando suas respostas:

(a) Ler o relógio de tempo real do hardware.

Creio que é necessário fazer uma chamada de sistema tendo em vista que isso é uma operação que faz acesso direto com o hardware.

(b) Enviar um pacote através da rede.

Sim, é necessário executar uma chamada de sistema, tendo em vista que para realizar uma operação através da rede é necessário acessar a placa de rede (acesso ao hardware).

(c) Calcular uma exponenciação.

Não é necessário fazer uma chamada de sistema, tendo em vista que essa é uma operação aritmética normal da CPU.

(d) Preencher uma área de memória do processo com zeros.

Não é necessário desde que essa escrita seja na mesma área do processo. O modo usuário já é suficiente.

(e) Remover um arquivo do disco.

É necessário que ocorra uma chamada de sistema, tendo em vista que sistema de arquivos é algo compartilhado e crítico. Se qualquer um puder remover quando quiser, existem grandes chances de erros e complicações ao sistema.

8 Questão 8

Coloque na ordem correta as ações abaixo, que ocorrem durante a execução da função `printf("Hello world")` por um processo (observe que nem todas as ações indicadas fazem parte da sequência).

Ordem correta:

- (h) O processo chama a função `printf` da biblioteca C.
- (c) A função de biblioteca `printf` recebe e processa os parâmetros de entrada (a string "Hello world").
- (d) A função de biblioteca `printf` prepara os registradores para solicitar a chamada de sistema `write()`.
- (g) Uma interrupção de software é acionada.
- (a) A rotina de tratamento da interrupção de software é ativada dentro do núcleo.
- (i) A operação de escrita no terminal é efetuada ou agendada pela rotina de tratamento da interrupção.
- (j) O controle volta para a função `printf` em modo usuário.
- (b) A função `printf` finaliza sua execução e devolve o controle ao código do processo.

Não fazem parte do processo:

- (e) O disco rígido gera uma interrupção indicando a conclusão da operação.
- (f) O escalonador escolhe o processo mais prioritário para execução.

Atividades

Atividade 1

O utilitário `strace` do UNIX permite observar a sequência de chamadas de sistema efetuadas por uma aplicação. Em um terminal, execute `strace date` para descobrir quais os arquivos abertos pela execução do utilitário `date` (que indica a data e hora correntes). Por que o utilitário `date` precisa fazer chamadas de sistema?

O utilitário `date` precisa fazer chamadas de sistema pois, para saber a data e hora correntes, é necessário um acesso ao hardware (relógio do sistema).

Atividade 2

O utilitário `ltrace` do UNIX permite observar a sequência de chamadas de biblioteca efetuadas por uma aplicação. Em um terminal, execute `ltrace date` para descobrir as funções de biblioteca chamadas pela execução do utilitário `date` (que indica a data e hora correntes). Pode ser observada alguma relação entre as chamadas de biblioteca e as chamadas de sistema observadas no item anterior?

Sim, pode ser observada uma relação direta. As chamadas de biblioteca servem como interface de alto nível que, internamente, realizam as chamadas de sistema necessárias para acessar os recursos do hardware e do núcleo do sistema operacional.